

LAPORAN JOBSHEET PEMROGRAMAN BERBASIS FRAMEWORK



Nama : Yan Daffa Putra Liandhie

NIM: 2341720142

Kelas: TI 3E

POLITEKNIK NEGERI MALANG TAHUN 2026

A. Langkah Kerja Praktikum

Langkah 1 – Menjalankan Project

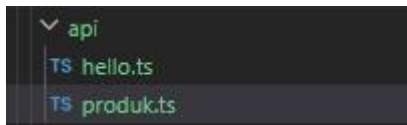
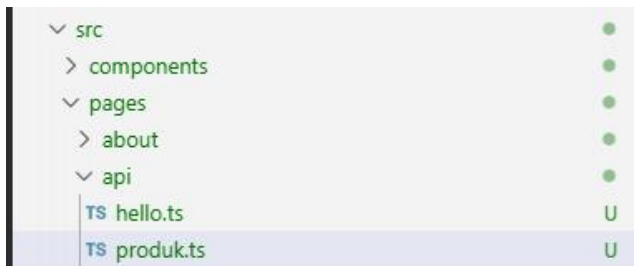
- npm run dev

Akses:

- http://localhost:3000

Langkah 2 – Membuat API Produk

1. Buat file pada pages/api/produk.js



2. Tambahkan data statis:

```
6.praktikum-ApiRoutes > my-app > src > pages > api > TS produk.ts > ...
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiResponse } from "next";
3
4 type Data = {
5   status: boolean;
6   status_code: number;
7   data: {
8     id: string;
9     nama: string;
10    harga: number;
11    ukuran: string;
12    warna: string;
13  }[];
14 };
15
16 export default function handler(
17   req: NextApiResponse,
18   res: NextApiResponse<Data>,
19 ) {
20   {
21     const data =
22     [
23       {
24         id: "1",
25         nama: "Kaos Polos",
26         harga: 10000,
27         ukuran: "L",
28         warna: "merah"
29       },
30       {
31         id: "2",
32         nama: "Kaos Berlengan Panjang",
33         harga: 15000,
34         ukuran: "M",
35         warna: "biru"
36       },
37     ];
38     res.status(200).json({status:true,status_code:200,data:data});
39   }
40 }
```

```
src > pages > api > TS produkts > handler > data
1 //Next.js API route support https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiResponse, NextApiRequest } from 'next'
3
4 type Data = {
5   status: boolean;
6   statusCode: number;
7
8   data: {
9     id: string;
10    nama: string;
11    harga: number;
12    ukuran: string;
13    warna: string;
14  }[];
15 };
16 export default function handler(
17   req: NextApiRequest,
18   res: NextApiResponse<Data>
19 ) {
20   const data = [
21     {
22       id: '1',
23       nama: 'Kaos Polos',
24       harga: 10000,
25       ukuran: 'L',
26       warna: 'merah'
27     },
28     {
29       id: '2',
30       nama: 'Kaos Berlengan panjang',
31       harga: 15000,
32       ukuran: 'M',
33       warna: 'biru'
34     }
35   ];
36   res.status(200).json({
37     status: true,
38     statusCode: 200,
39     data: data
40   });
41 }
```

3. Akses: <http://localhost:3000/api/produk>

```
localhost:3000/api/produk
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "1",
      "nama": "Kaos Polos",
      "harga": 10000,
      "ukuran": "L",
      "warna": "merah"
    },
    {
      "id": "2",
      "nama": "Kaos Berlengan Panjang",
      "harga": 15000,
      "ukuran": "M",
      "warna": "biru"
    }
  ]
}
```

```
Pretty-print ✓

{
  "status": true,
  "statusCode": 200,
  "data": [
    {
      "id": "1",
      "nama": "Kaos Polos",
      "harga": 10000,
      "ukuran": "L",
      "warna": "merah"
    },
    {
      "id": "2",
      "nama": "Kaos Berlengan panjang",
      "harga": 15000,
      "ukuran": "M",
      "warna": "biru"
    }
  ]
}
```

Langkah 3 – Fetch Data API di Frontend

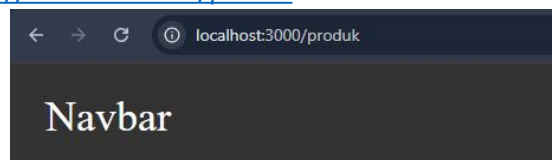
1. Buka pages/product/index.tsx

- Modifikasi kode
 - Tambahkan useEffect() dan comment useEffect untuk isLogin

```
6.praktikum-APIRoutes > my-app > src > pages > produk > index.tsx > kategori
1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3
4  Windsurf: Refactor | Explain
5  type ProductType =
6  {
7    id: string;
8    nama: string;
9    harga: number;
10   ukuran: string;
11   warna: string;
12 }
13
14 Windsurf: Refactor | Explain | Generate JSDoc | X
15 const kategori = () => {
16   // const [isLogin, setIsLogin] = useState(false);
17   // const { push } = useRouter();
18   const [products, setProducts] = useState([]);
19
20   // useEffect(() => {
21   //   if (!isLogin) {
22   //     push("/auth/login");
23   //   }
24   // }, []);
25
26   useEffect(() => {
27     fetch("/api/produk")
28       .then((response) => response.json())
29       .then((responseData) => {
30         //console.log("Data produk:", responseData.data);
31         setProducts(responseData.data);
32       })
33       .catch((error) => {
34         console.error("Error fetching produk:", error);
35       });
36   }, []);
37
38   return (
39     <div>
40       <h1>Daftar Produk</h1>
41       {products.map((product: ProductType) => (
42         <div key={product.id}>
43           <h2>{product.nama}</h2>
44           <p>Harga: {product.harga}</p>
45           <p>Ukuran: {product.ukuran}</p>
46           <p>Warna: {product.warna}</p>
47         </div>
48       ))}
49     </div>
50   );
51 }
52 export default kategori;
```

```
TS produk.ts U index.tsx U X
src > pages > produk > index.tsx > kategori
1 import {useRouter} from "next/router";
2 import { useEffect, useState } from "react";
3
4 type ProductType = {
5   id: string;
6   nama: string;
7   harga: number;
8   ukuran: string;
9   warna: string;
10 }
11
12 const kategori = () => {
13   const [products, setProducts] = useState([]);
14
15   useEffect(() => {
16     fetch('/api/produk')
17       .then((response) => response.json())
18       .then((responseData) => {
19         setProducts(responseData.data);
20       })
21       .catch((error) => {
22         console.error('Error fetching data:', error);
23       });
24   }, []);
25
26   return (
27     <div>
28       <h1>Daftar Produk</h1>
29       {products.map((product: ProductType) => (
30         <div key={product.id}>
31           <h2>{product.nama}</h2>
32           <p>Harga: {product.harga}</p>
33           <p>Ukuran: {product.ukuran}</p>
34           <p>Warna: {product.warna}</p>
35         </div>
36       ))}
37     </div>
38   );
39 };
40 export default kategori;
41
42
```

2. Jalankan browser <http://localhost:3000/produk>



Daftar Produk

Kaos Polos

Harga: 10000

Ukuran: L

Warna: merah

Kaos Berlengan Panjang

Harga: 15000

Ukuran: M

Warna: biru

Navbar

Daftar Produk

Kaos Polos

Harga: 10000

Ukuran: L

Warna: merah

Kaos Berlengan panjang

Harga: 15000

Ukuran: M

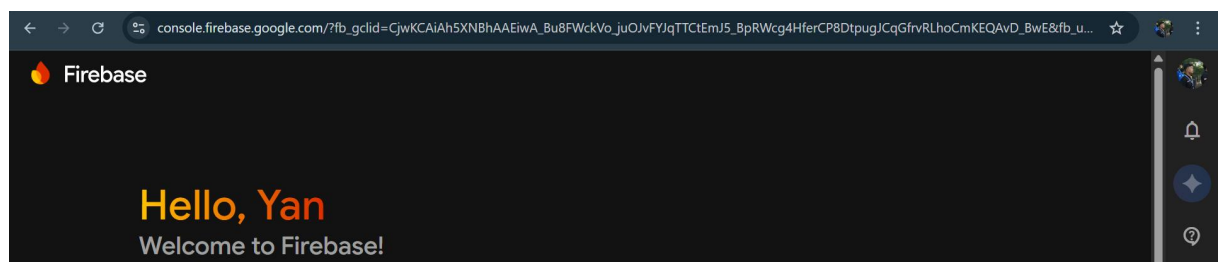
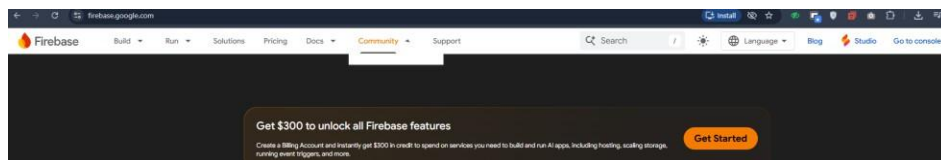
Warna: biru

footer

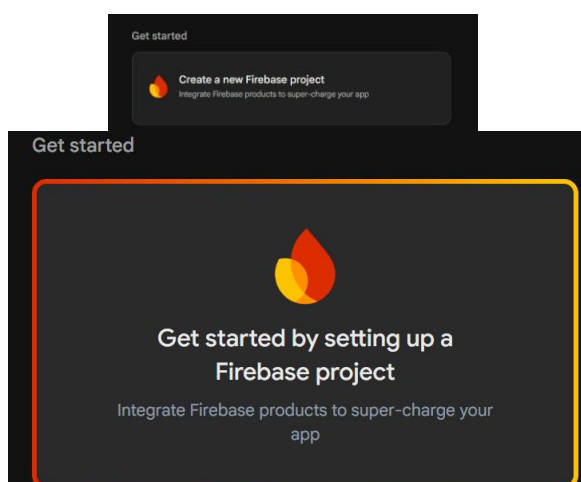
B. Integrasi Firebase

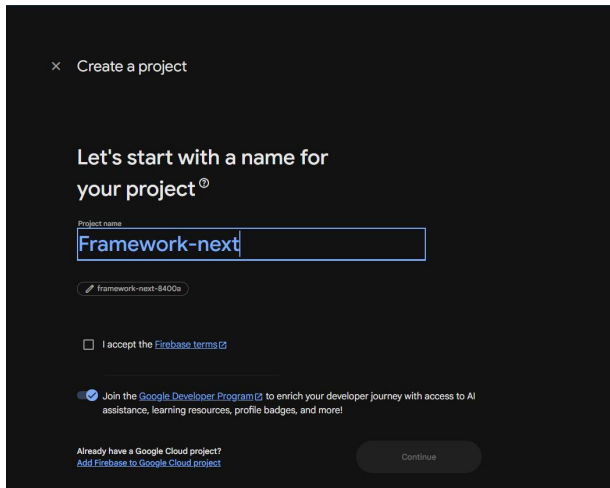
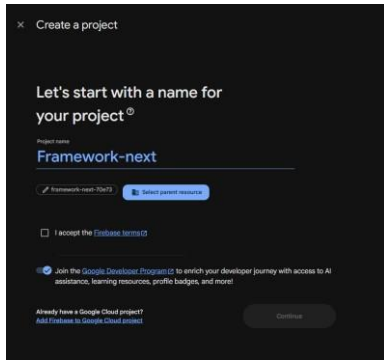
Langkah 5 – Setup Firebase

1. Buka Firebase Go To Console (login dengan login google)

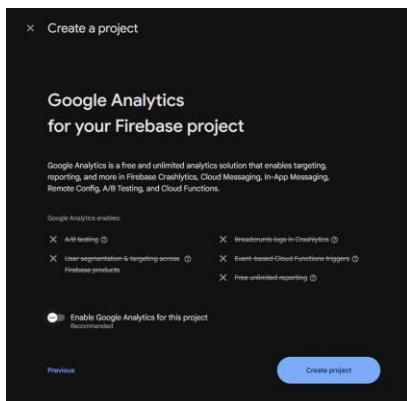


2. Buat project baru

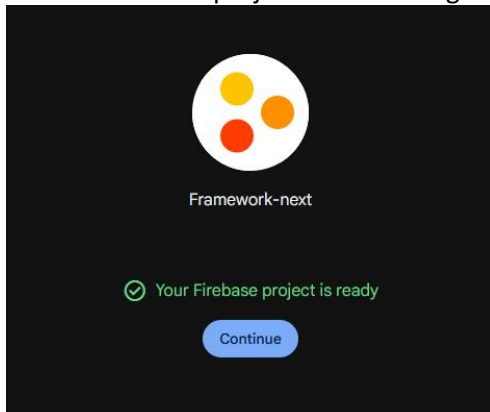


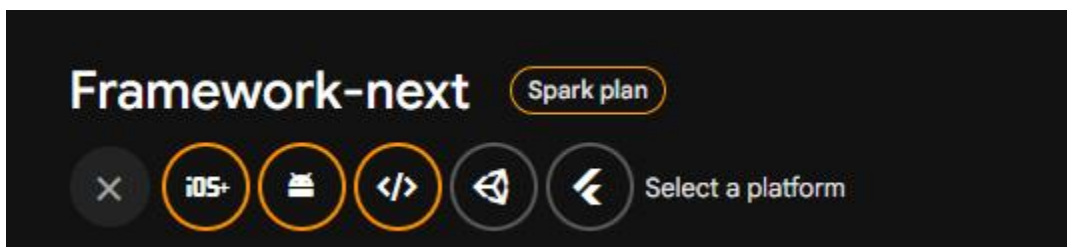
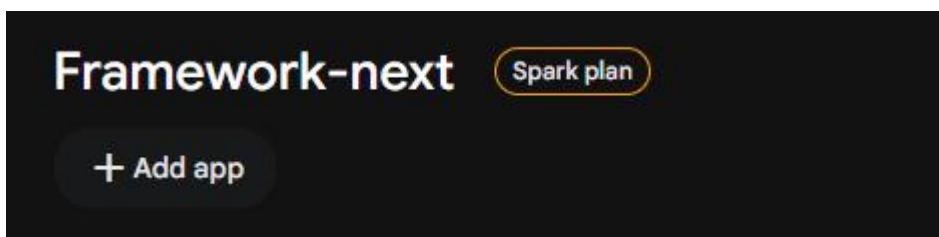
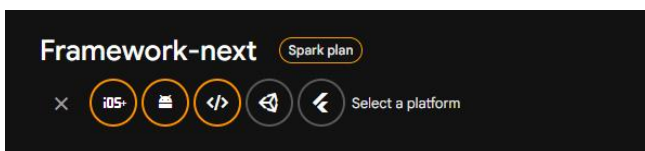
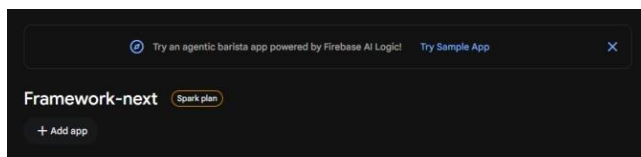
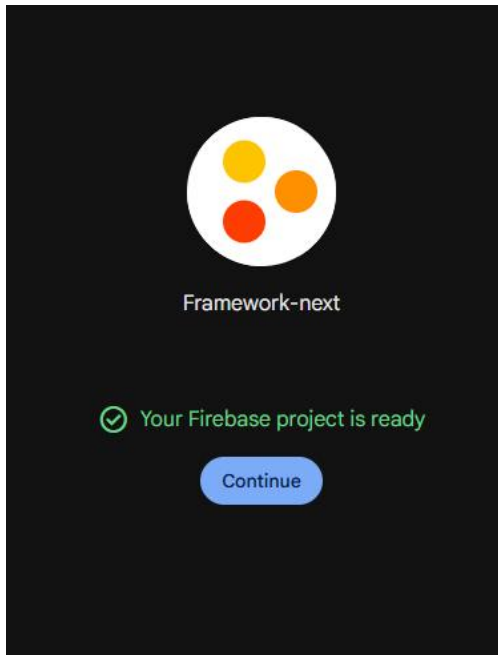


Note : jangan lupa select parent resource nya

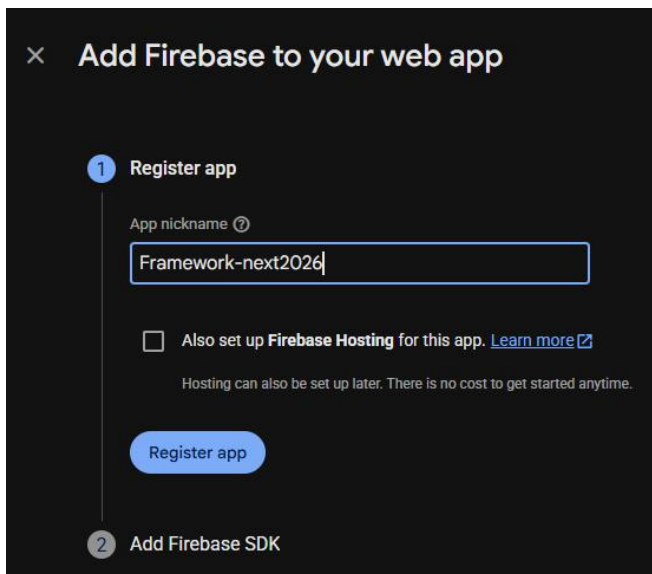
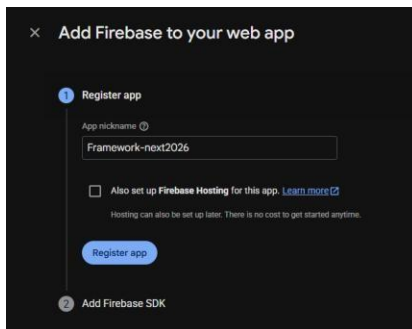


Note : klik create project dan disable google analytics

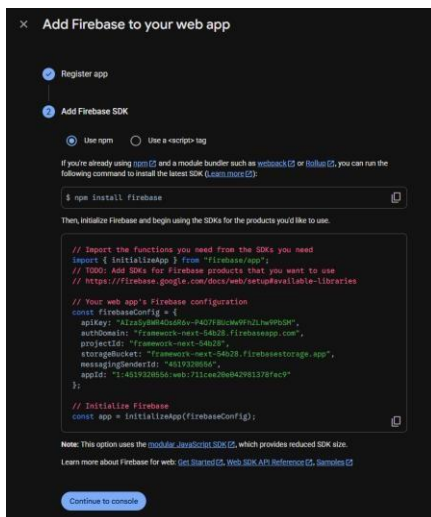




Note klik add app dan pilih web



Note : klik register app



Note : klik continue to console

×

Add Firebase to your web app

✓

Register app

2

Add Firebase SDK

☒ Use npm

☐ Use a <script> tag

If you're already using [npm](#) and a module bundler such as [webpack](#) or [Rollup](#), you can run the following command to install the latest SDK ([Learn more](#)):

\$ npm install firebase

Then, initialize Firebase and begin using the SDKs for the products you'd like to use.

```
// Import the functions you need from the SDKs you need
import { initializeApp } from "firebase/app";
import { getAnalytics } from "firebase/analytics";
// TODO: Add SDKs for Firebase products that you want to use
// https://firebase.google.com/docs/web/setup#available-libraries

// Your web app's Firebase configuration
// For Firebase JS SDK v7.20.0 and later, measurementId is optional
const firebaseConfig = {
  apiKey: "AIzaSyBRxkbFQH9YfyxuFhzT6EA-5XuSy9yvYU",
  authDomain: "framework-next-5a45d.firebaseio.com",
  projectId: "framework-next-5a45d",
  storageBucket: "framework-next-5a45d.firebaseio.com",
  messagingSenderId: "406057713087",
  appId: "1:406057713087:web:c749da06f1525fc14db50a",
  measurementId: "G-FZDCL2JF6"
};

// Initialize Firebase
const app = initializeApp(firebaseConfig);
const analytics = getAnalytics(app);
```

Note: This option uses the [modular JavaScript SDK](#), which provides reduced SDK size.

Learn more about Firebase for web: [Get Started](#), [Web SDK API Reference](#), [Samples](#)

Continue to console

Aktifkan Firestore Database

Build

App Check

App Hosting

Authentication

Data Connect

Extensions

Firestore Database

Functions

Hosting

Machine Learning

Phone Verificati... NEW

Realtime Database

Storage

Run

Analytics

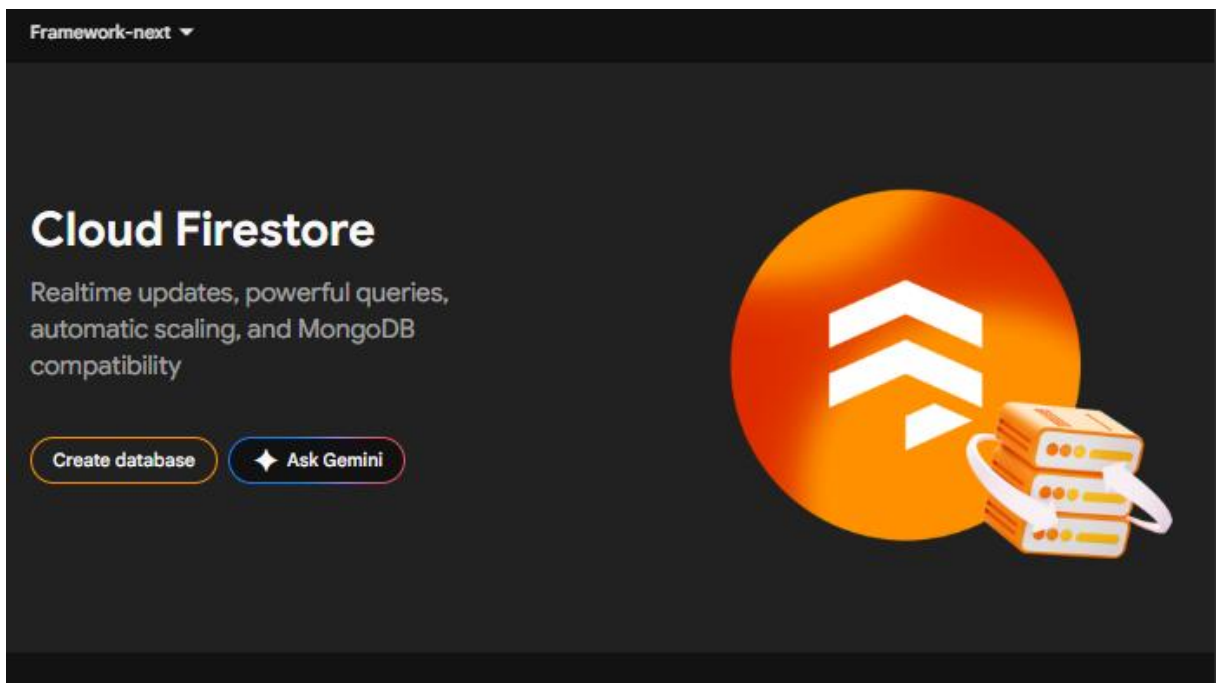
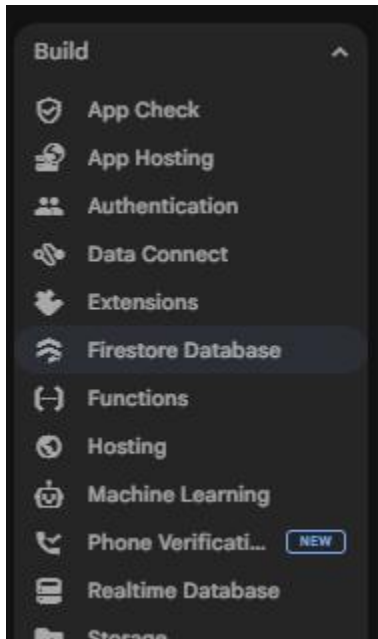
AI

Cloud Firestore

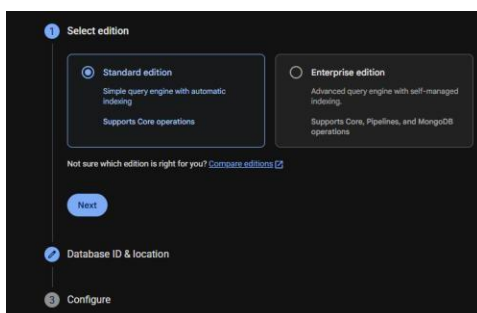
Realtime updates, powerful queries, automatic scaling, and MongoDB compatibility

Create database

Ask Gemini



Note : klik create database



1 Select edition

2 Database ID & location

3 Configure

Database ID

(default)

Location

asia-southeast2 (Jakarta)

ⓘ Your location setting is where your Cloud Firestore data will be stored

1 After you set this location, you cannot change it later

Learn more

Next

×

Create a database

1 Select edition

2 Database ID & location

3 Configure

☒ Standard edition

Simple query engine with automatic indexing

Supports Core operations

☐ Enterprise edition

Advanced query engine with self-managed indexing

Supports Core, Pipelines, and MongoDB operations

Not sure which edition is right for you? [Compare editions](#)

Next

×

Create a database

1 Select edition

2 Database ID & location

3 Configure

Database ID

(default)

Location

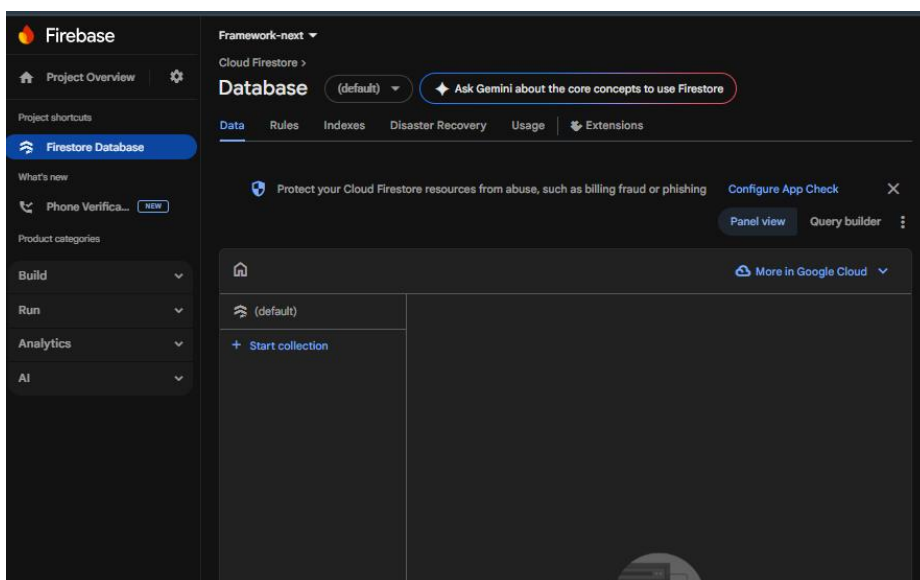
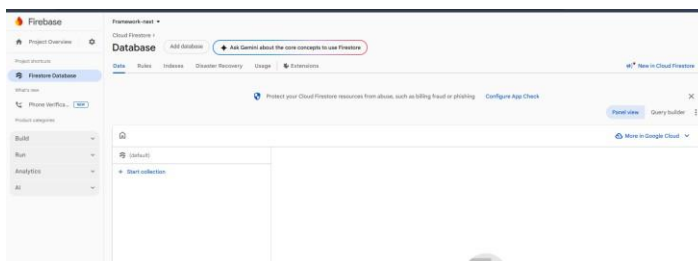
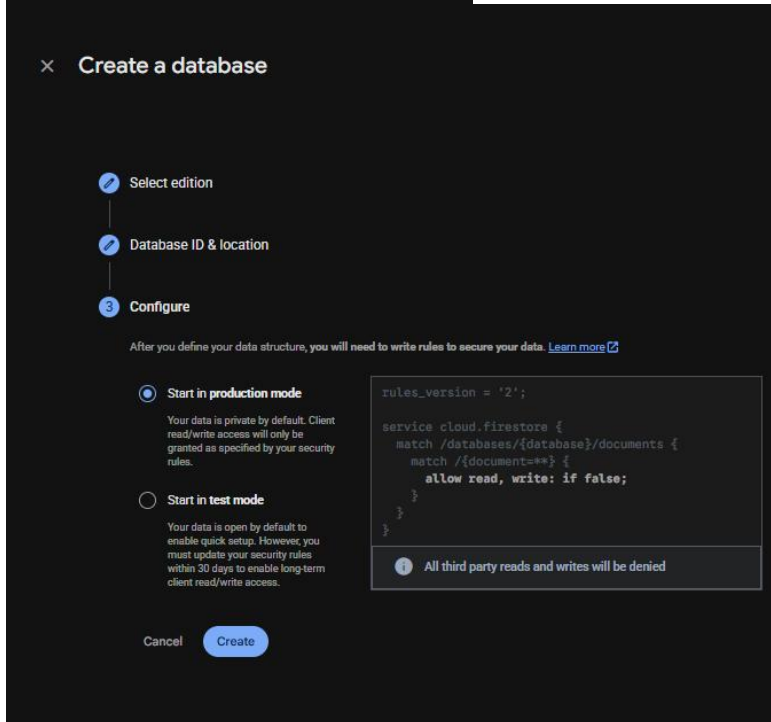
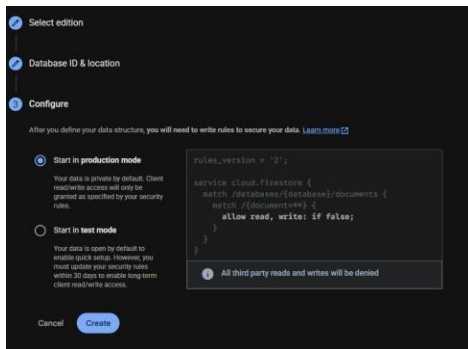
asia-southeast2 (Jakarta)

ⓘ Your location setting is where your Cloud Firestore data will be stored

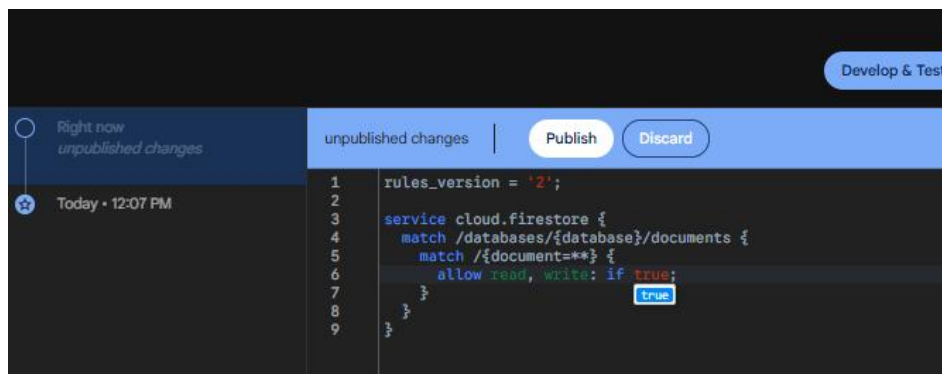
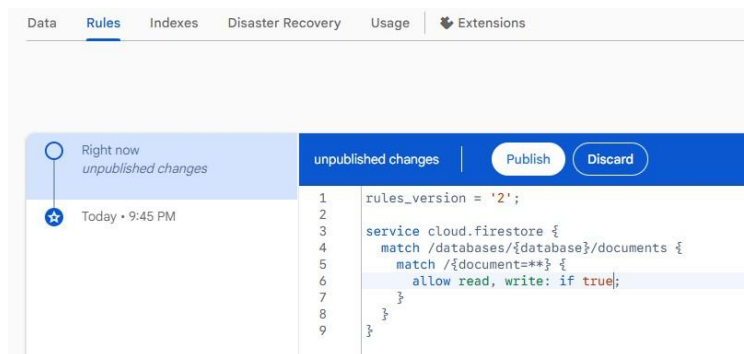
1 After you set this location, you cannot change it later

Learn more

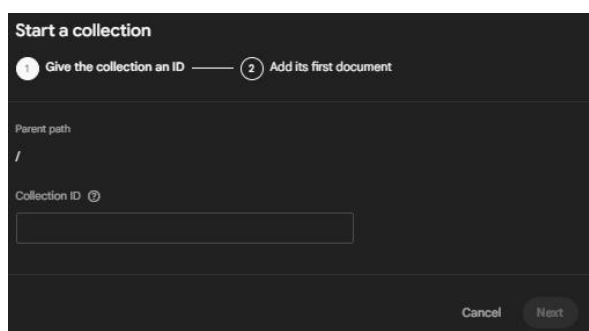
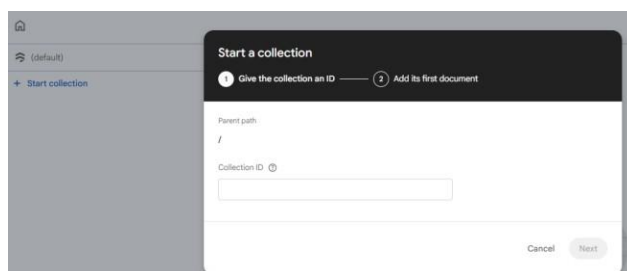
Next



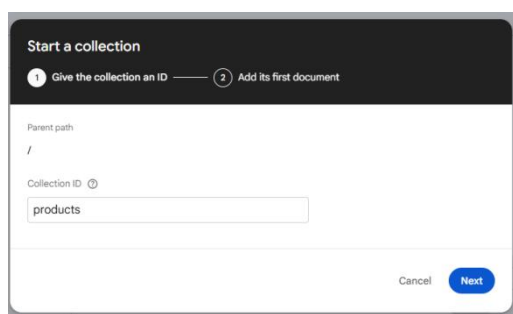
Rubah rulesnya jadi true dan klik publish



3. Buat collection:



4. products



Start a collection

1 Give the collection an ID — 2 Add its first document

Parent path
/

Collection ID ⓘ
products

Cancel Next

Gunakan auto-id

Start a collection

✓ Give the collection an ID — 2 Add its first document

Document parent path
/products

Document ID ⓘ
Auto-ID

Required

Field	Type
	string

String

... Add field

Cancel Save

Start a collection

✓ Give the collection an ID — 2 Add its first document

Document parent path
/products

Document ID ⓘ
Auto-ID

Required

Field	Type
	string

String

... Add field

Cancel Save

5. Tambahkan field:

Start a collection

Give the collection an ID

Add its first document

Document parent path

/products

Document ID ⓘ

WhGlAfdQjwlPerAJFGW

Field

Type

name = string

String

baju polo

Field

Type

Value

price = number

100000

Field

Type

size = string

String

xl

Add field

Cancel Save

Start a collection

Give the collection an ID

Add its first document

Document parent path

/products

Document ID ⓘ

whGlAfdQjwlPerAJFGW

Field

Type

name = string

String

baju polo

Field

Type

Value

price = number

100000

Field

Type

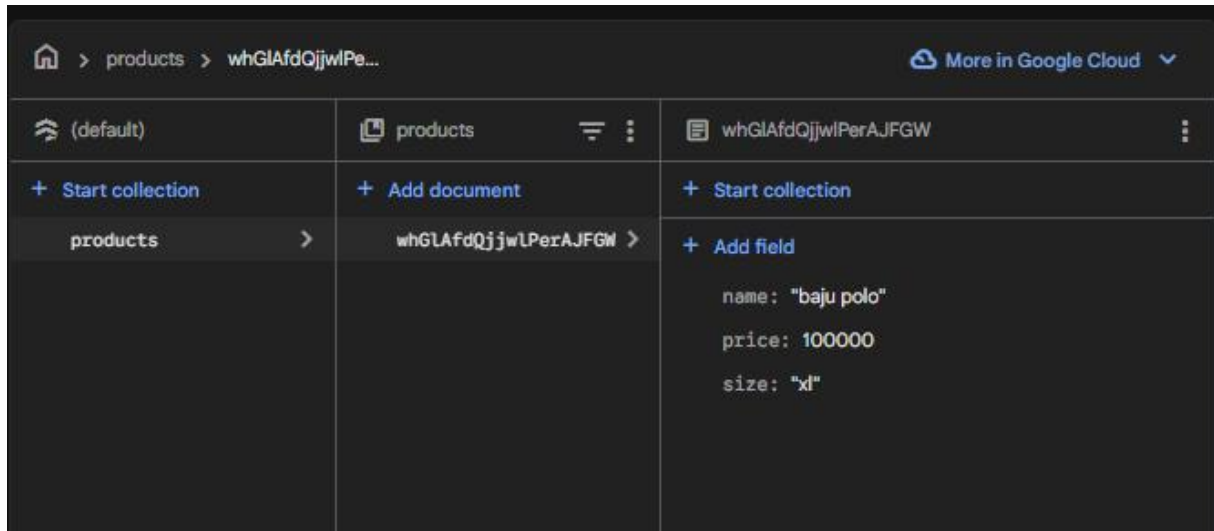
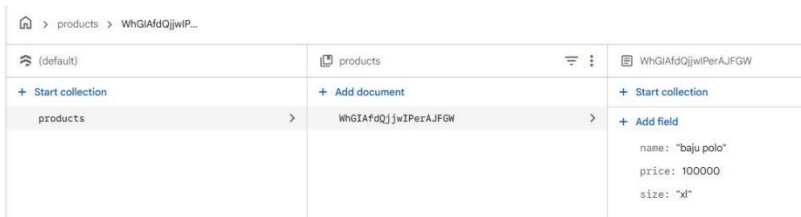
size = string

String

xl

Add field

Cancel Save



Langkah 6 – Install Firebase

1. npm install firebase

PS D:\1.MateriKuliah\2026-ganjil\PemogramanFramework\Code\6.praktikum-ApiRoutes\my-app> npm install firebase

added 79 packages, and audited 426 packages in 33s

142 packages are looking for funding
run `npm fund` for details

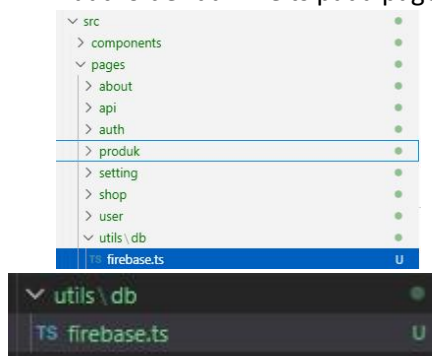
found 0 vulnerabilities

```
D:\tugas\semester 6\framework\js7\code\my-app>npm install firebase
```

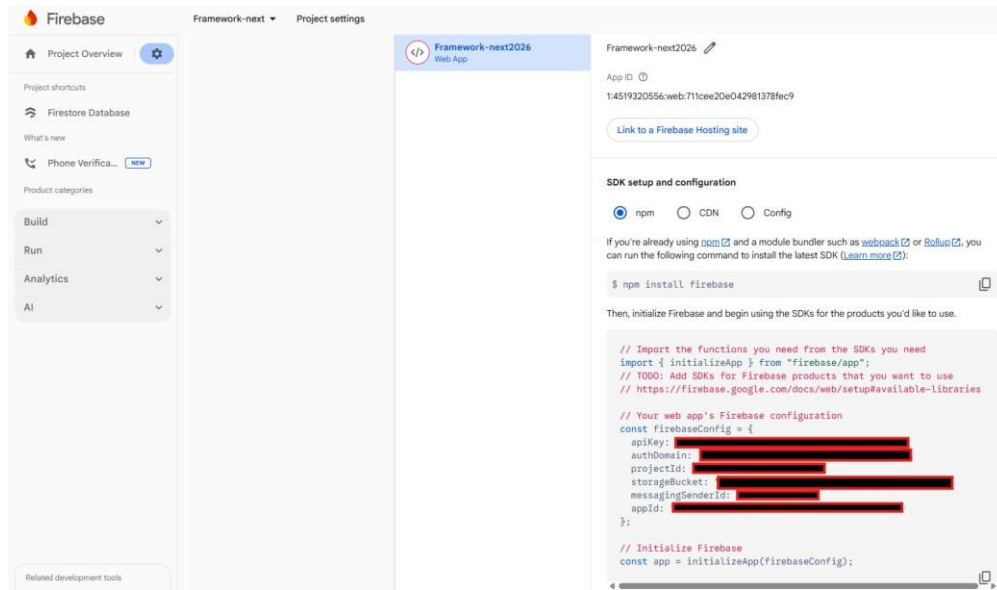
added 79 packages, and audited 468 packages in 22m

154 packages are looking for funding
run `npm fund` for details

2. Buat folder dan file ts pada pages/utlis/db/firebase.ts



3. Copy paste yang ada pada kotak merah ke file firebase.ts



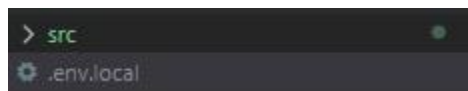
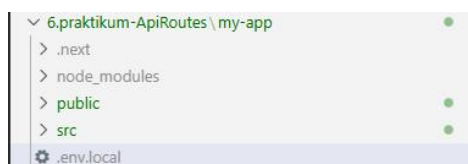
Langkah 7 – Konfigurasi Environment Variable agar credensial firebase tidak dapat dilihat saat dipush di repository

```

src > pages > utils > db > TS firebase.ts > ...
1 // Import the functions you need from the SDKs you need
2 import { initializeApp } from "firebase/app";
3 import { getAnalytics } from "firebase/analytics";
4 // TODO: Add SDKs for Firebase products that you want to use
5 // https://firebase.google.com/docs/web/setup#available-libraries
6
7 // Your web app's Firebase configuration
8 // For Firebase JS SDK v7.20.0 and later, measurementId is optional
9 const firebaseConfig = {
10   apiKey: "AIzaSyBRxkbQh0YyfyxufhzT6EA-5XuSy9yvYU",
11   authDomain: "framework-next-5a45d.firebaseio.com",
12   projectId: "framework-next-5a45d",
13   storageBucket: "framework-next-5a45d.firebaseio.com",
14   messagingSenderId: "406057713087",
15   appId: "1:406057713087:web:c749da06f1525fc14db50a",
16   measurementId: "G-FZDCCJZJF6"
17 };
18
19 // Initialize Firebase
20 const app = initializeApp(firebaseConfig);
21 const analytics = getAnalytics(app);

```

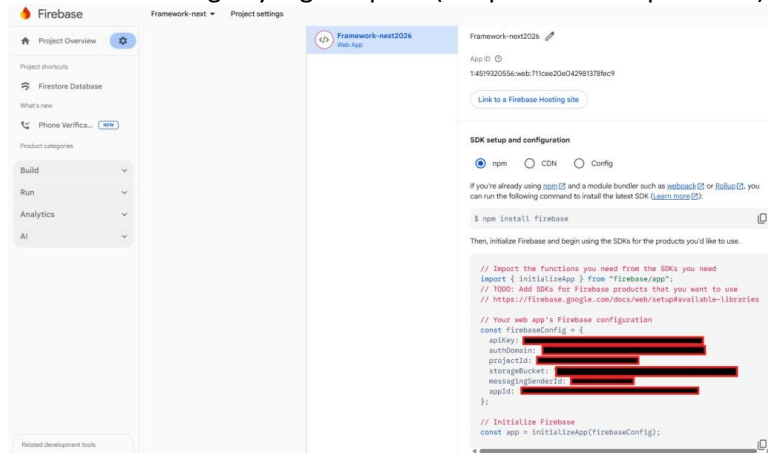
1. Buat file: .env.local



2. Modifikasi file env

- FIREBASE_API_KEY=xxxx, FIREBASE_AUTH_DOMAIN=xxxx, FIREBASE_PROJECT_ID=xxxx, FIREBASE_STORAGE_BUCKET=xxxx, FIREBASE_MESSAGING_SENDER_ID=xxxx, FIREBASE_APP_ID=xxxx

- Isikan sesuai dengan yang ada pada (tanpa koma dan petik dua)



6.praktikum-ApiRoutes > my-app > .env.local

```
1 FIREBASE_API_KEY=[REDACTED]
2 FIREBASE_AUTH_DOMAIN=[REDACTED]
3 FIREBASE_PROJECT_ID=[REDACTED]
4 FIREBASE_STORAGE_BUCKET=[REDACTED]
5 FIREBASE_MESSAGING_SENDER_ID=[REDACTED]
6 FIREBASE_APP_ID=[REDACTED]
7
```

```
.env.local
1 FIREBASE_API_KEY=AiZaSyA9n8sXo217m1v5z5z5z5z5z5z5z5z5
2 FIREBASE_AUTH_DOMAIN=framework-next-5a45d.firebaseio.com
3 FIREBASE_PROJECT_ID=framework-next-5a45d
4 FIREBASE_STORAGE_BUCKET=framework-next-5a45d.firebaseio.com
5 FIREBASE_MESSAGING_SENDER_ID=406057713087
6 FIREBASE_APP_ID=1:406057713087:web:c749da06f1525fc14db50a
```

Langkah 8 – Konfigurasi Firebase

1. Modifikasi firebase.ts

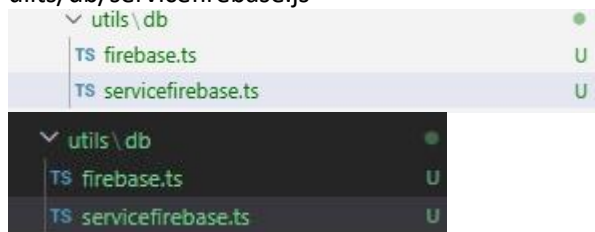
```
6.praktikum-ApiRoutes > my-app > src > pages > utils > db > TS firebase.ts > [0] default
1 // Import the functions you need from the SDKs you need
2 import { initializeApp } from "firebase/app";
3 // TODO: Add SDKs for Firebase products that you want to use
4 // https://firebase.google.com/docs/web/setup#available-libraries
5
6 // Your web app's Firebase configuration
7 const firebaseConfig = {
8   apiKey: process.env.FIREBASE_API_KEY,
9   authDomain: process.env.FIREBASE_AUTH_DOMAIN,
10  projectId: process.env.FIREBASE_PROJECT_ID,
11  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
12  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
13  appId: process.env.FIREBASE_APP_ID
14 };
15
16 // Initialize Firebase
17 const app = initializeApp(firebaseConfig);
18
19 export default app;
```

```
src > pages > utils > db > TS firebase.ts > ...
1 // Import the functions you need from the SDKs you need
2 import { initializeApp } from "firebase/app";
3 // TODO: Add SDKs for Firebase products that you want to use
4 // https://firebase.google.com/docs/web/setup#available-libraries
5
6 // Your web app's Firebase configuration
7 // For Firebase JS SDK v7.20.0 and later, measurementId is optional
8 const firebaseConfig = {
9   apiKey: process.env.FIREBASE_API_KEY,
10  authDomain: process.env.FIREBASE_AUTH_DOMAIN,
11  projectId: process.env.FIREBASE_PROJECT_ID,
12  storageBucket: process.env.FIREBASE_STORAGE_BUCKET,
13  messagingSenderId: process.env.FIREBASE_MESSAGING_SENDER_ID,
14  appId: process.env.FIREBASE_APP_ID,
15 };
16
17 // Initialize Firebase
18 const app = initializeApp(firebaseConfig);
19
20 export default app;
```

Langkah 9 – Ambil Data dari Firestore

1. Buat file

- utils/db/servicefirebase.js



- modifikasi file servicefirebase.js

```
6.praktikum-APIRoutes > my-app > src > pages > utils > db > TS servicefirebase.ts > ...
1 import { getFirestore, collection, getDocs } from "firebase/firestore";
2 import app from "../firebase";
3
4 const db = getFirestore(app);
5
6 Windsurf: Refactor | Explain | Generate JSDoc | X
7 export async function retrieveProducts(collectionName: string) {
8   const snapshot = await getDocs(collection(db, collectionName));
9   const data = snapshot.docs.map((doc) => ({
10     id: doc.id,
11     ...doc.data(),
12   }));
13   return data;
14 }
```



```
src > pages > utils > db > TS servicefirebase.ts > retrieveProducts
1 import {getFirestore, collection, getDocs} from 'firebase/firestore';
2 import app from '../firebase';
3
4 const db = getFirestore(app);
Windsurf: Refactor | Explain | Generate JSDoc | X
5 export async function retrieveProducts(collectionName: string) {
6   const snapshot = await getDocs(collection(db, collectionName));
7   const data = snapshot.docs.map((doc) => ({ id: doc.id, ...doc.data() }));
8   return data;
9 }
```

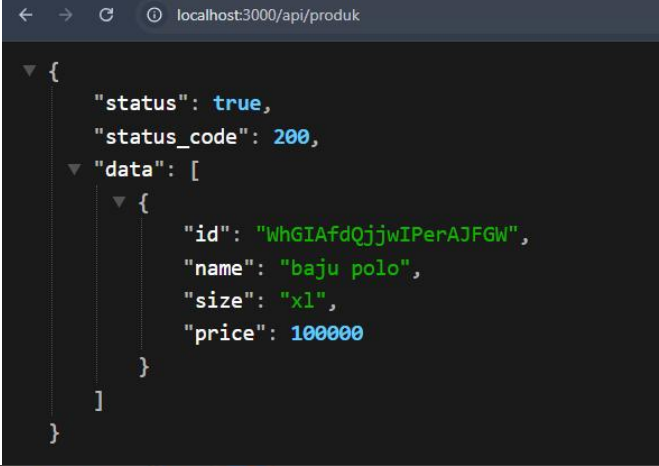
Langkah 10 – API Mengambil Data Firebase

1. Edit pages/api/product.js:

```
6.praktikum-APIRoutes > my-app > src > pages > api > TS produk.ts > ...
1 // Next.js API route support: https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiRequest, NextApiResponse } from 'next';
3 import { retrieveProducts } from '../utils/db/servicefirebase';
4
Windsurf: Refactor | Explain
5 type Data = {
6   status: boolean;
7   status_code: number;
8   data: any;
9 };
10
Windsurf: Refactor | Explain | Generate JSDoc | X
11 export default async function handler(
12   req: NextApiRequest,
13   res: NextApiResponse<Data>,
14 ) {
15   const data = await retrieveProducts("products");
16   res.status(200).json({ status: true, status_code: 200, data });
17 }
18
```

```
TS produk.ts U X index.tsx U TS next-env.d.ts TS firebase.ts U TS servicefirebase.ts U .env.local
src > pages > api > TS produk.ts > handler
1 //Next.js API route support https://nextjs.org/docs/api-routes/introduction
2 import type { NextApiRequest, NextApiResponse } from 'next'
3 import { retrieveProducts } from '../utils/db/servicefirebase';
4
Windsurf: Refactor | Explain
5 type Data = {
6   status: boolean;
7   status_code: number;
8   data: any;
9 };
Windsurf: Refactor | Explain | Generate JSDoc | X
10 export default function handler(
11   req: NextApiRequest,
12   res: NextApiResponse<Data>
13 ) {
14   const data = retrieveProducts("products")
15   res.status(200).json({
16     status: true,
17     status_code: 200,
18     data });
19 }
20
```

2. Jalankan browser <http://localhost:3000/api/produk>



```
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "WhGIAfdQjjwIPerAJFGW",
      "name": "baju polo",
      "size": "xl",
      "price": 100000
    }
  ]
}
```

Pretty-print ☒

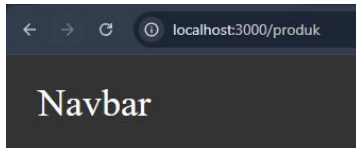
```
{
  "status": true,
  "status_code": 200,
  "data": [
    {
      "id": "WhGIAfdQjjwIPerAJFGW",
      "size": "xl",
      "name": "baju polo",
      "price": 100000
    }
  ]
}
```

Modifikasi index.ts pada produk sesuaikan nama typenya dan db nya

```
6.praktikum-ApiRoutes > my-app > src > pages > produk > index.tsx > kategori > useEffect() callback > then() callback
1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3
4  Windsurf: Refactor | Explain
5  type ProductType =
6  {
7    id: string;
8    name: string;
9    price: number;
10   size: string;
11 }
12
13 Windsurf: Refactor | Explain | Generate JSDoc | X
14 const kategori = () => {
15   // const [isLogin, setIsLogin] = useState(false);
16   // const { push } = useRouter();
17   const [products, setProducts] = useState([]);
18
19   // useEffect(() => {
20   //   if (!isLogin) {
21   //     push("/auth/login");
22   //   }
23   // }, []);
24
25   useEffect(() => {
26     fetch("/api/produk")
27       .then((response) => response.json())
28       .then((responseData) => {
29         setProducts(responseData.data);
30         // console.log("Data produk:", responseData.data);
31       })
32       .catch((error) => {
33         console.error("Error fetching produk:", error);
34       });
35   }, []);
36
37   return (
38     <div>
39       <h1>Daftar Produk</h1>
40       {products.map((product: ProductType) => (
41         <div key={product.id}>
42           <h2>{product.name}</h2>
43           <p>Harga: {product.price}</p>
44           <p>Ukuran: {product.size}</p>
45         </div>
46       ))}
47     </div>
48   );
49 };
50 export default kategori;
```

```
src > pages > produk > index.tsx > kategori > products.map() callback
1  import { useRouter } from "next/router";
2  import { useEffect, useState } from "react";
3
4  Windsurf: Refactor | Explain
5  type ProductType = {
6    id: string;
7    nama: string;
8    price: number;
9    size: string;
10 }
11
12 Windsurf: Refactor | Explain | Generate JSDoc | X
13 const kategori = () => {
14   const [productcs, setProducts] = useState([]);
15
16   useEffect(() => {
17     fetch('/api/produk')
18       .then((response) => response.json())
19       .then((responseData) => {
20         setProducts(responseData.data);
21       })
22       .catch((error) => {
23         console.error('Error fetching data:', error);
24       });
25   }, []);
26
27   return (
28     <div>
29       <h1>Daftar Produk</h1>
30       {productcs.map((product: ProductType) => (
31         <div key={product.id}>
32           <h2>{product.nama}</h2>
33           <p>Harga: {product.price}</p>
34           <p>Ukuran: {product.size}</p>
35         </div>
36       ))}
37     </div>
38   );
39 };
40 export default kategori;
```


Jalankan browser

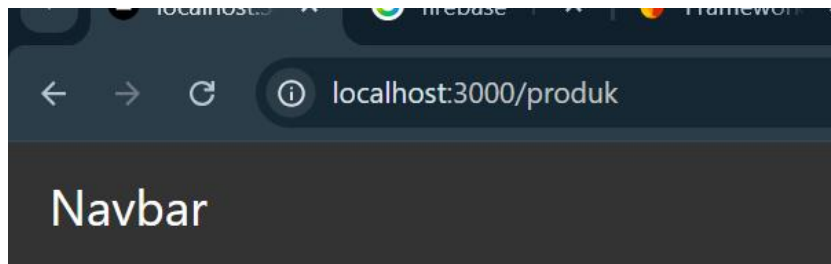


Daftar Produk

baju polo

Harga: 100000

Ukuran: xl



Daftar Produk

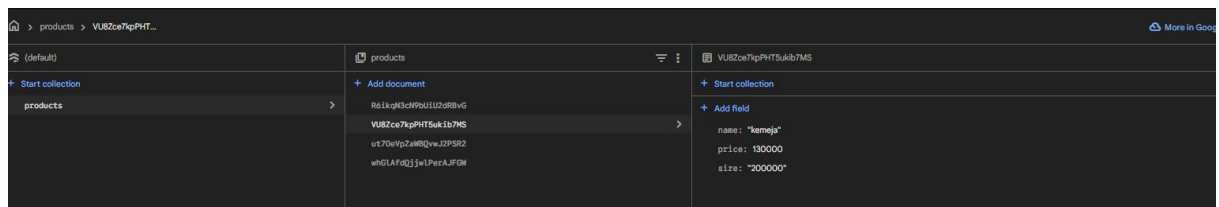
Harga: 100000

Ukuran: xl

C.Tugas Praktikum

Tugas 1 (Wajib)

- Tambahkan minimal 3 data produk di Firestore



- Pastikan data tampil di halaman produk

Navbar

Daftar Produk

jaket

Harga: 200000

Ukuran: XL

kemeja

Harga: 130000

Ukuran: 200000

celana

Harga: 150000

Ukuran: L

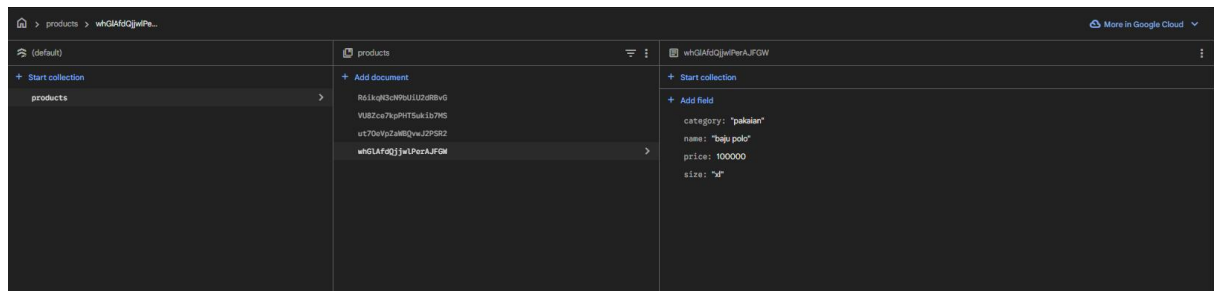
baju polo

Harga: 100000

Ukuran: xl

Tugas 2 (Wajib)

- Tambahkan field baru:
 - Category



- Tampilkan category di frontend

```

src > pages > produk > index.tsx > kategori > products.map() callback
1 import {useRouter} from "next/router";
2 import {useEffect, useState} from "react";
3
Windsurf: Refactor | Explain
4 type ProductType = {
5   id: string;
6   name: string;
7   price: number;
8   size: string;
9   category: string;
10 }
11
Windsurf: Refactor | Explain | Generate JSDoc | X
12 const kategori = () => {
13   const [products, setProducts] = useState([]);
14
15   useEffect(() => {
16     fetch('/api/produk')
17       .then((response) => response.json())
18       .then((responseData) => {
19         setProducts(response.data);
20       })
21       .catch((error) => {
22         console.error('Error fetching data:', error);
23       });
24   }, []);
25
26   return (
27     <div>
28       <h1>Daftar Produk</h1>
29       {products.map((product: ProductType) => (
30         <div key={product.id}>
31           <h2>{product.name}</h2>
32           <p>Harga: {product.price}</p>
33           <p>Ukuran: {product.size}</p>
34           <p>Kategori: {product.category}</p>
35         </div>
36       ))}
37     </div>
38   );
39 };
40 export default kategori;
41

```

Navbar

Daftar Produk

varsity

Harga: 200000

Ukuran: XL

Kategori: jaket

kemeja

Harga: 130000

Ukuran: 200000

Kategori: pakaian

celana

Harga: 150000

Ukuran: L

Kategori: celana

baju polo

Harga: 100000

Ukuran: xl

Kategori: pakaian

- Tambahkan tombol Refresh Data
- Gunakan fetch ulang tanpa reload halaman

```

1  < pages > produk > @ index > @ ProductType
2  import { useState, useEffect } from 'react';
3
4  type ProductType = {
5    id: string;
6    name: string;
7    price: number;
8    size: string;
9    category: string;
10 };
11
12 const Kategori = () => {
13   const [products, setProducts] = useState<ProductType[]>([]);
14   const [loading, setLoading] = useState(false);
15
16   const fetchData = async () => {
17     try {
18       setLoading(true);
19       const response = await fetch('/api/produk');
20       const responseData = await response.json();
21       setProducts(response.data);
22     } catch (error) {
23       console.error('Error fetching data:', error);
24     } finally {
25       setLoading(false);
26     }
27   };
28
29   useEffect(() => {
30     fetchData();
31   }, []);
32
33   return (
34     <div>
35       <h1>Daftar Produk</h1>
36
37       <button
38         onClick={fetchData}
39         style={{
40           marginBottom: '20px',
41           padding: '10px 20px',
42           backgroundColor: '#007bff',
43           color: 'white',
44           border: 'none',
45           borderRadius: '5px',
46           cursor: 'pointer',
47         }}
48       > {loading ? 'Loading...' : 'Refresh'}
49     </button>
50
51     {products.map((product) => (
52       <div key={product.id}>
53         <h2>{product.name}</h2>
54         <p>Harga: {product.price}</p>
55         <p>Ukuran: {product.size}</p>
56         <p>Kategori: {product.category}</p>
57       </div>
58     ))}
59   );
60 };
61
62 export default Kategori;

```

Daftar Produk

Refresh

varsity

Harga: 200000

Ukuran: XL

Kategori: jaket

kemeja

Harga: 130000

Ukuran: 200000

Kategori: pakaian

celana

Harga: 150000

Ukuran: L

Kategori: celana

baju polo

Harga: 100000

Ukuran: xl

Kategori: pakaian

D. Pertanyaan Evaluasi

1. Apa fungsi API Routes pada Next.js?

Jawab : untuk membuat backend end point langsung di dalam project Next.js tanpa memerlukan servr terpisah.

2. Mengapa .env.local tidak boleh di-push ke repository?

Jawab :karena di dalam .env.local terdapat data data seperti API key, database URL, Secret token yang dapat disalahgunakan

3. Apa perbedaan data statis dan data dinamis?

Jawab : data statis merupakan data yang tida berubah dan tidak bergantung pada interaksi pengguna, sementara data dinamis adalah data yang dapat berubah ubah sewaktu waktu

4. Mengapa Next.js disebut framework fullstack?

Jawab : karena dalam Next.js bisa menangani pengembangan frontend dan backend dalam satu framework

Github : https://github.com/YanDaffaPutraLiandhie/PBF_2026